

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

	organized.			
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System

CO3-ASSESSMENT 2

Course:

Software Engineering / Code Review and Version Control Evaluation

Name:

Davis Damien M

192521124

1. Problem Overview

1.1 Problem Statement

Hotels often manage reservations, room allocation, housekeeping, maintenance requests, guest communication, billing, and personalized services using multiple disconnected systems. This leads to inefficient operations, delayed service delivery, poor resource utilization, and inconsistent guest experiences.

The Intelligent AI-Based Smart Hotel Operations and Guest Experience Management System aims to integrate hotel operations into a unified platform enhanced with AI-driven automation and decision support.

1.2 Implemented Solution

The reviewed project provides a centralized web-based hotel management platform with AI-assisted features such as:

- Intelligent room recommendation
- Automated guest service requests
- Housekeeping task optimization
- Predictive maintenance alerts
- Occupancy and revenue analytics
- AI chatbot for guest assistance
- Personalized guest experience management

The system uses a client-server architecture with a web frontend, backend API, database layer, and AI modules for analytics and automation.

1.3 Project Objectives

- Improve operational efficiency
- Enhance guest satisfaction
- Automate repetitive hotel workflows
- Provide real-time analytics and reporting

- Enable intelligent decision-making using AI

1.4 Key Functionalities

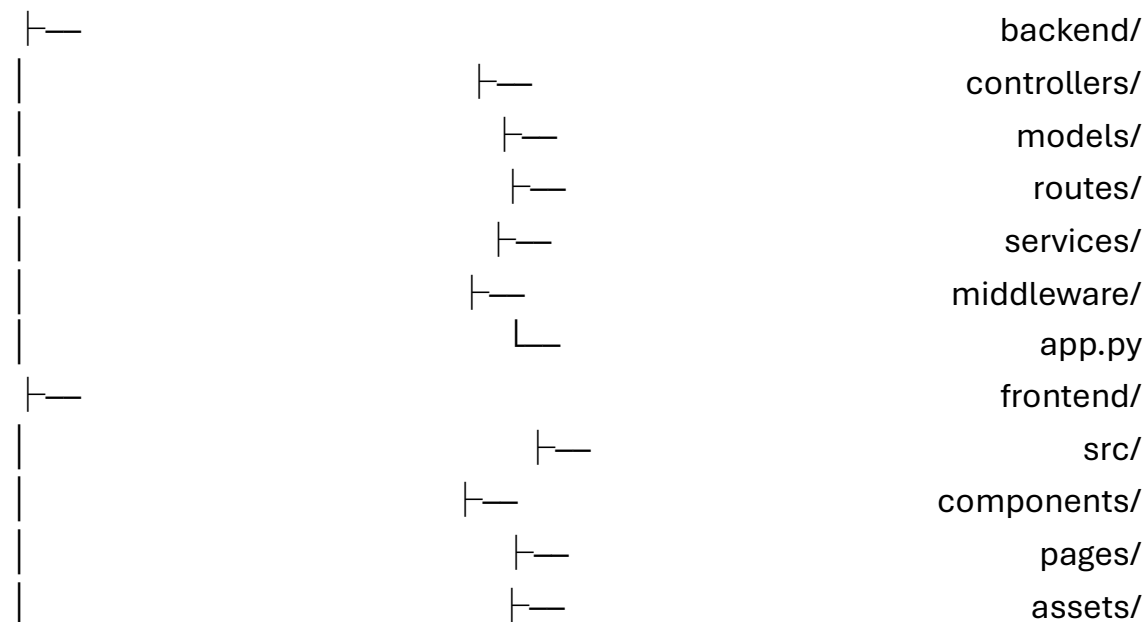
Module	Functionality
Reservation	Booking, cancellation, modification
Room Management	Availability, allocation, room status
Housekeeping	Task assignment and tracking
Guest Services	Requests, complaints, concierge support
Billing	Invoice generation and payment tracking
AI Engine	Recommendations, chatbot, predictive analytics
Dashboard	Occupancy, revenue, service metrics

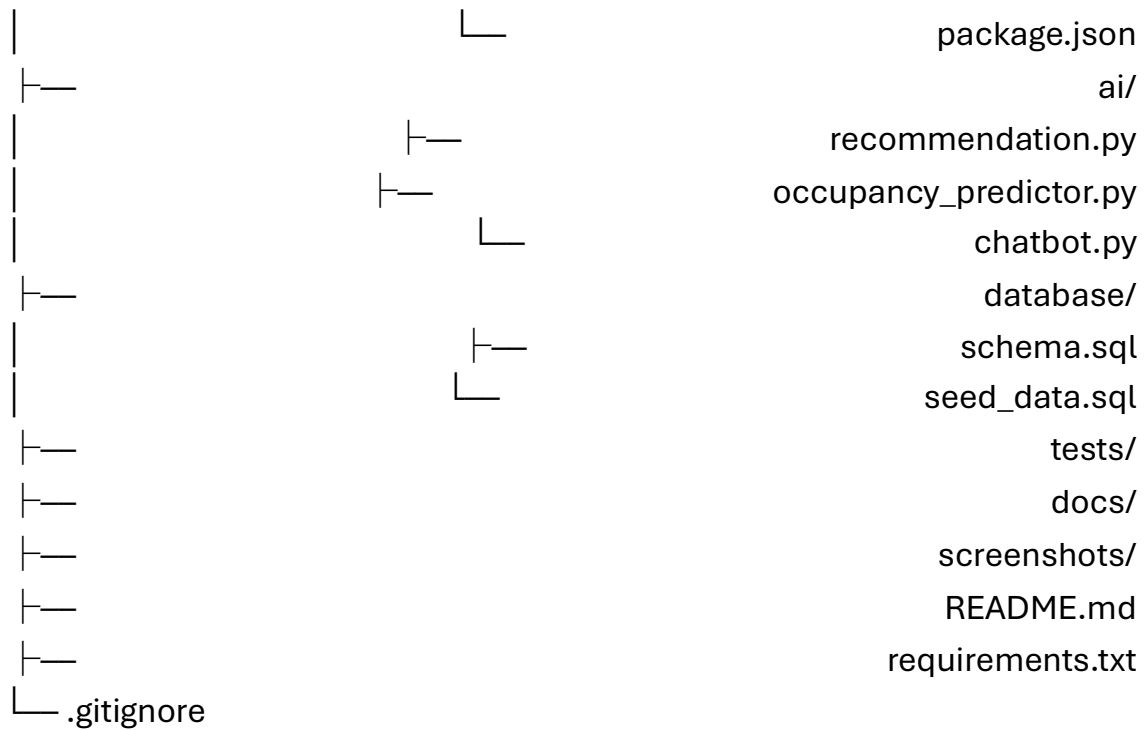
2. Repository Organization

2.1 Repository Structure

Example repository layout:

project-root/





2.2 Evaluation

Strengths

- Clear separation of frontend, backend, AI, and database components.
- Modular folder organization.
- Logical grouping of source files.
- Independent testing directory.
- Documentation stored separately.

Weaknesses

- Some utility functions are duplicated.
- Configuration files are scattered.
- AI scripts could be packaged as a reusable module.

2.3 Maintainability

The repository supports maintainability by:

- Isolating business logic from presentation logic.

- Organizing routes and controllers separately.
- Keeping AI models independent from web components.
- Providing a dedicated tests directory.

This structure enables multiple developers to work simultaneously with minimal merge conflicts.

3. Code Quality Review

3.1 Readability

Strengths

- Meaningful variable names
- Descriptive function names
- Consistent indentation
- Appropriate use of comments
- Modular functions

Example:

```
def recommend_room(guest_profile):
    available_rooms = get_available_rooms()
    return rank_rooms(available_rooms, guest_profile)
```

The function name clearly expresses its purpose.

3.2 Modularity

The application follows modular design principles.

Example modules:

- reservation_service.py
- housekeeping_service.py
- billing_service.py
- ai_recommendation.py

Advantages:

- Reusability
- Easier debugging
- Independent testing
- Better scalability

3.3 Coding Standards

The code largely follows standard practices:

- PEP 8 (Python)
- Consistent naming conventions
- CamelCase for classes
- snake_case for variables and functions

Areas for Improvement

- Some functions exceed 50–80 lines.
- Hardcoded configuration values appear in source files.
- Exception handling is inconsistent.

Example improvement:

Current:

```
price = room.price * 1.18
```

Improved:

GST_RATE	=	1.18
price = room.price * GST_RATE		

3.4 Documentation

Most public functions contain docstrings.

Example:

```
def assign_housekeeping(room_id):
```

